

NOW WE WILL ADD CODE FOR ABOUTUS AND CONTACTUS MODULE

Create AboutUS name ka empty controllers

for AboutUS controller index method code

-----INDEX.CSHTML-----

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>About Us - LibraryManagement</title>

    <!-- Add your Bootstrap CDN here if not already included -->

    <link href="https://jsdelivr.net" rel="stylesheet">

    <style>

        body {

            font-family: sans-serif;

            background-color: #f8f9fa;

            margin: 0;

            padding: 0;

        }
```

```
.navbar-custom {  
  
    background-color: #212529;  
  
    padding: 10px 20px;  
  
}
```

```
.navbar-custom .navbar-brand, .navbar-custom .nav-link {  
  
    color: #fff !important;  
  
}
```

```
.navbar-custom .nav-link:hover {  
  
    color: #ccc !important;  
  
}
```

```
.content-container {  
  
    background: #fff;  
  
    border-radius: 4px;  
  
    box-shadow: 0 1px 3px rgba(0,0,0,0.1);  
  
    padding: 40px;  
  
    margin-top: 30px;  
  
    margin-bottom: 50px;  
  
}
```

```
.footer-custom {  
  
    background-color: #212529;  
  
    color: #aaa;  
  
    text-align: center;  
  
    padding: 20px 0;  
  
    font-size: 14px;  
  
    position: fixed;  
  
    bottom: 0;  
  
    width: 100%;  
  
}
```

```
body {  
  
    padding-bottom: 60px;  
  
}
```

```
/* Prevents content from being hidden behind fixed footer */
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<!-- Main Content Section -->
```

```
<div class="container">
```

```
<div class="content-container">
```

```
<h2 class="mb-4 fw-bold text-dark">About Us</h2>
```

```
<p class="text-muted lead">Welcome to LibraryManagement, your ultimate portal for digital and physical resource tracking.</p>
```

```
<p class="text-secondary">Our mission is to simplify school and university cataloging by providing a seamless, fast, and secure database solution for students and administrators alike. Manage books, check availability times, and handle borrowing requests with just a few clicks.</p>
```

```
<div class="row mt-5">
```

```
<div class="col-md-4">
```

```
<h5 class="fw-bold">Our Vision</h5>
```

```
<p class="text-muted small">To build connected environments where academic resources are accessible to everyone, instantly.</p>
```

```
</div>
```

```
<div class="col-md-4">
```

```
<h5 class="fw-bold">Smart Tracking</h5>
```

```
<p class="text-muted small">Monitor real-time item checkouts, pending return dates, and stock limits automatically.</p>
```

```
</div>
```

```
<div class="col-md-4">
```

```
<h5 class="fw-bold">Easy Access</h5>
```

```
<p class="text-muted small">A clean interface optimized for desktop, tablet, and mobile screens alike.</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

Create ContactUS name ka empty controllers

for ContactUS controller index method code

-----INDEX.CDHTML-----

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Contact Us - LibraryManagement</title>
```

```
<!-- Add your Bootstrap CDN here if not already included -->
```

```
<link href="https://jsdelivr.net" rel="stylesheet">
```

```
<style>
```

```
    body {
```

```
        font-family: sans-serif;
```

```
        background-color: #f8f9fa;
```

```
        margin: 0;
```

```
        padding: 0;
```

```
    }
```

```
    .navbar-custom {
```

```
        background-color: #212529;
```

```
        padding: 10px 20px;
```

```
    }
```

```
    .navbar-custom .navbar-brand, .navbar-custom .nav-link {
```

```
        color: #fff !important;
```

```
    }
```

```
    .navbar-custom .nav-link:hover {
```

```
        color: #ccc !important;

    }
}
```

```
.content-container {

    background: #fff;

    border-radius: 4px;

    box-shadow: 0 1px 3px rgba(0,0,0,0.1);

    padding: 40px;

    margin-top: 30px;

    margin-bottom: 50px;

}
```

```
.btn-submit {

    background-color: #0d6efd;

    color: white;

    border: none;

    font-weight: 500;

}
```

```
.btn-submit:hover {

    background-color: #0b5ed7;
```

```
}
```

```
.footer-custom {  
  
    background-color: #212529;  
  
    color: #aaa;  
  
    text-align: center;  
  
    padding: 20px 0;  
  
    font-size: 14px;  
  
    position: fixed;  
  
    bottom: 0;  
  
    width: 100%;  
  
}
```

```
body {  
  
    padding-bottom: 60px;  
  
}
```

```
/* Prevents content from being hidden behind fixed footer */
```

```
</style>
```

```
</head>
```

```
<body>
```



```
<!-- Main Content Section -->

<div class="container">

  <div class="content-container">

    <div class="row">

      <!-- Form Column -->

        <div class="col-md-7 pe-md-5">

          <h2 class="mb-4 fw-bold text-dark">Contact Us</h2>

          <form action="#" method="POST">

            <div class="mb-3">

              <label for="name" class="form-label text-secondary small fw-bold">Full
Name</label>

              <input type="text" class="form-control form-control-sm rounded-1" id="name"
required placeholder="Enter your name">

            </div>

            <div class="mb-3">

              <label for="email" class="form-label text-secondary small fw-bold">Email
Address</label>

              <input type="email" class="form-control form-control-sm rounded-1"
id="email" required placeholder="Enter your email">

            </div>

            <div class="mb-3">

              <label for="message" class="form-label text-secondary small fw-bold">Your
Message</label>
```

```
<textarea class="form-control form-control-sm rounded-1" id="message"
rows="4" required placeholder="Type your message here..."></textarea>
```

```
</div>
```

```
<button type="submit" class="btn btn-sm btn-submit px-4 rounded-1">Submit
Ticket</button>
```

```
</form>
```

```
</div>
```

```
<!-- Info Column -->
```

```
<div class="col-md-5 mt-5 mt-md-0 border-start ps-md-4">
```

```
<h4 class="fw-bold mb-3">Support Desk</h4>
```

```
<p class="text-muted small">Have issues with broken links, missing inventory, or
user accounts? Drop a message and our support admin panel team will reach out within 24
hours.</p>
```

```
<hr class="my-4 text-muted">
```

```
<h6 class="fw-bold mb-1">Email</h6>
```

```
<p class="text-muted small mb-3">support@librarymanagement.com</p>
```

```
<h6 class="fw-bold mb-1">Hours</h6>
```

```
<p class="text-muted small">Mon - Fri: 9:00 AM - 6:00 PM</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

NOW RUN THE APPLICATION

YOU WILL GET ABOUTUS PAGE

YOU WILL GET CONTACT US PAGE

-----UNIT TESTING -----

FOR BOOK MODULE

run your project and see book details then start

To thoroughly test your **Book Module**, you should write unit tests for the controller actions handling your catalog view (**Index**)

you can do and new data entries (**Create**).

This ensures your UI tables, search parameters, and pagination logic function correctly without breaking.

Here are the step-by-step instructions and code to implement unit tests using **xUnit**, **FluentAssertions** in ASP.NET Core MVC.

Step 1: Set Up Your Unit Test Project

1. Right-click your solution in Visual Studio → **Add** → **New Project**.
2. Select **xUnit Test Project (.NET Core)** and click next. GIVE NAME TO PROJECT
3. Open your package manager console and add these required testing dependencies:
shell

```
dotnet add package FluentAssertions
```

```
dotnet add package Microsoft.AspNetCore.Mvc.Core
```

FluentAssertions is a popular open-source .NET library that replaces traditional unit test verification methods with a natural, highly readable English-like syntax.

Instead of relying on rigid, hard-to-read commands bundled with default test frameworks (like `Assert.AreEqual(expected, actual)`),

Fluent Assertions uses extension methods starting with `.Should()` to clearly express test intent. It works alongside top test frameworks like [xUnit](#), NUnit, and MSTest

Right-click your test project dependencies → **Add Project Reference** → Select your core MVC Web Application project (`LMSYSTEM`).

Step 3: Implement the Unit Tests

Create a class named `BooksControllerTests.cs` in your test project to validate that your index page retrieves titles, populates view items, and redirects bad input validation cycles.

csharp

```
using FluentAssertions;
using LMSystem.Controllers;
using LMSystem.Interfaces;
using LMSystem.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
```

```
namespace TestProject1
{
    public class BooksControllerInMemoryTests:IDisposable
```

```

{
    private readonly LibraryContext _context;
    private readonly BooksController _controller;

    public BooksControllerInMemoryTests()
    {
        // 1. Build an isolated database configuration inside system RAM memory
        var options = new DbContextOptionsBuilder<LibraryContext>()
            .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString()) // Unique
seed name per test run
            .Options;

        _context = new LibraryContext(options);
        _context.Database.EnsureCreated();

        // 2. Seed test records matching your exact user interface display state
        SeedDatabase();

        // 3. Instantiate your actual production controller using the local context
        _controller = new BooksController(_context);
    }

    private void SeedDatabase()
    {
        _context.Books13.AddRange(new List<Book>
        {
            new Book { BookId = 1, Title = "bootstrap", Author = "amir", ISBN = "888-
0201616224", PublishedDate = DateTime.Parse("2026-07-24"), IsAvailable = true },
            new Book { BookId = 2, Title = "node js", Author = "shadab", ISBN = "888-
0201616224", PublishedDate = DateTime.Parse("2026-07-18"), IsAvailable= true },
            new Book { BookId = 3, Title = "software engineerig", Author = "raju", ISBN =
"888-0201616224", PublishedDate = DateTime.Parse("2026-07-24"), IsAvailable = true }
        });
        _context.SaveChanges();
    }

    // =====
    // TEST 1: VERIFY SEARCH TERM FILTERS
    // =====
    [Fact]
    public async Task Index_FiltersBooks_WhenSearchStringIsProvided()
    {
        // Act - Pass a target search parameter "node" down to the endpoint
        var result = await _controller.Index(searchQuery: "node", page: 1);

        // Assert - Evaluate payload contents
        var viewResult = result.Should().BeOfType<ViewResult>().Subject;
        var model =
viewResult.Model.Should().BeAssignableTo<IEnumerable<Book>>().Subject;

        // Only "node js" should be extracted from our database block
        model.Should().ContainSingle();
        model.First().Title.Should().Be("node js");
    }
}

```

```

    }

    // =====
    // TEST 2: VERIFY PAGINATION LOGIC LIMITS
    // =====
    [Fact]
    public async Task Index_ReturnsCorrectPageSize_ForPaginatedRequests()
    {
        // Setup - Assuming your production controller splits data to 2 rows max per
screen
        // Act - Request page number 2
        var result = await _controller.Index(searchQuery: null, page: 2);

        // Assert
        var viewResult = result.Should().BeOfType<ViewResult>().Subject;
        var model =
viewResult.Model.Should().BeAssignableTo<IEnumerable<Book>>().Subject;

        // Since we have 3 seeded items, page 1 holds 2 items and page 2 holds exactly
1 trailing item
        model.Count().Should().Be(1);
    }

    public void Dispose()
    {
        // Wipe instance structures on suite finish
        _context.Database.EnsureDeleted();
        _context.Dispose();
    }
}
}

```

Step 4: Run Your Test Log

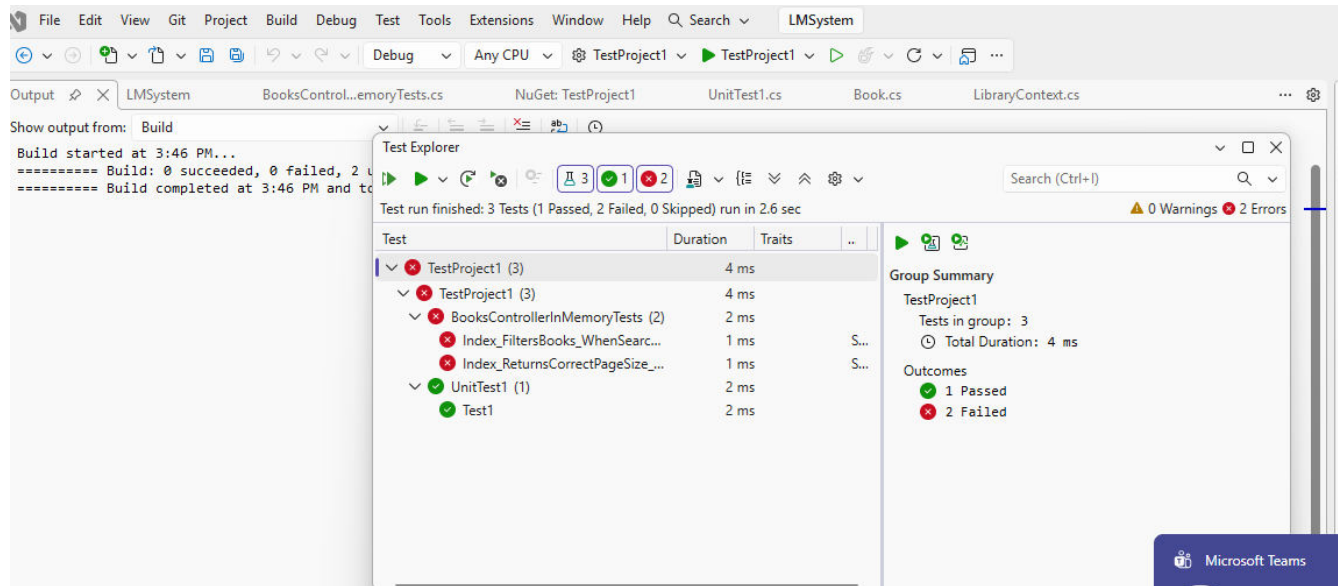
1. In Visual Studio, go to the top navbar and click **Test** → **Run All Tests** (or press `Ctrl + R, A`).
2. Open the **Test Explorer** panel to verify your test suites execute and pass successfully.

Step 1: Install the InMemory Test Dependency

Open your Package Manager Console inside your Test Project and install the specialized EF Core memory runner:

shell

```
dotnet add package Microsoft.EntityFrameworkCore.InMemory
```



we have 1 record

1 page 5

2 page per 5

Step 3: Implement Your Production Controller Logic (Reference Structure)

For the tests above to execute properly, verify your production code inside `BooksController.cs` uses LINQ queries structured like this:

csharp


```
public async Task<IActionResult> Index(string searchString, int pageNumber = 1)
{
    int pageSize = 2; // Matches our pagination requirement step
    var query = _context.Books.AsQueryable();

    // Handle incoming search string
    if (!string.IsNullOrEmpty(searchString))
    {
        query = query.Where(b => b.Title.Contains(searchString) ||
b.Author.Contains(searchString));
    }

    // Execute slice processing
    var paginatedList = await query
        .Skip((pageNumber - 1) * pageSize)
        .Take(pageSize)
        .ToListAsync();

    return View(paginatedList);
}
```

Use code with caution.

Press **Ctrl + R**,

now again run the test cases

<https://github.com/omache/Library-Management-Service-App>

Library Management System

If we think lms as enterprise level

Project Overview

The **Library Management System** is a comprehensive web application built with ASP.NET Core MVC, Entity Framework, and MSSQL. It enables the efficient management of a library's operations and services through an intuitive interface for three distinct roles: Administrator, Librarian, and Member. This project demonstrates proficiency in full-stack development, with a focus on clean architecture, database management, and user experience.

Features

Administrator

- Full System Access:** Administrators have complete control over the system, including the ability to manage books, authors, publishers, categories, and users (Librarians & Members).
- User Management:** Set limits on the number of books a member can issue and the number of times a book can be re-issued.
- Book Management:** Add, edit, and delete books, authors, publishers, and categories/genres.

Librarian

- Book Issuance:** Issue books to members, respecting individual member limits.
- Returns and Fines:** Manage book returns and calculate/accept overdue fines.
- Reporting:** Generate reports for all books, authors, publishers, categories, issued books, and overdue books.
- Search and Availability:** Search for books by title, author, publisher, or category, and check the availability of copies.
- Return Date Extension:** Extend book return dates upon request, with validation against other member requests.

Member

- Book Search:** Search for books by title, author, publisher, or category, and view available copies.
- Book Requests:** Request to issue or return a book, adhering to member limits.
- Account Management:** View issued books, unpaid overdues, and history of transactions. Edit profile information, upload a profile picture, and request account deletion.
- Profile Management:** Members can update their personal details and password.

Technologies Used

- Backend:** ASP.NET Core MVC, C#
- Frontend:** HTML5, CSS3, JavaScript

- Database:** MSSQL with Entity Framework Core
- IDE:** Visual Studio

Getting Started

Prerequisites

- [.NET SDK](#) (6.0 or later)
- [SQL Server](#) (Express or higher)
- [Visual Studio 2022](#) (with ASP.NET and web development workload)

visual studio

Add-Migration MIG1

Update-Database

1. Use the following default credentials to log in as an Administrator, Librarian, or Member:

- Administrator:** [admin@example.com](#) / Password123
user
- Librarian:** [librarian@example.com](#) / Password123
- Password123
- Member:** [member@example.com](#) / Password123

Technologies Used

- ASP.NET Core MVC:** Backend and frontend web development.

- Entity Framework Core**: ORM for database management.
- MSSQL**: Database management system.
- HTML/CSS**: Styling and structure of the frontend.
- JavaScript**: Dynamic and interactive user experience.

Project Structure

- Models**: Defines the data structures (e.g., Books, Authors, Users).
 - Controllers**: Manages the application logic and handles user input.
 - Views**: Renders the user interface.
 - Data**: Handles database context and migrations.
-

```
SELECT * FROM BOOK
```

```
SELECT * FROM Borrow
```

```
SELECT * FROM AspNetRoleClaims
```

```
SELECT * FROM AspNetRoles
```

```
SELECT * FROM AspNetUserClaims
```

```
SELECT * FROM AspNetUserLogins
```

```
SELECT * FROM AspNetUserRoles
```

```
SELECT * FROM AspNetUsers
```

```
SELECT * FROM AspNetUserTokens
```

Based on your database schema image and SQL tables, you are running an **ASP.NET Core Identity** authentication back-end linked to your library tables (`BOOK` and `Borrow`).

Here are the precise `INSERT` SQL statements to populate your roles and map your target users (`Administrator`, `Librarian`, and `Member`) directly into your database.

1. Insert Identity Roles

First, insert the required roles into the `AspNetRoles` table. We pre-generate uppercase normalized fields so ASP.NET Identity can find them.

sql

```
INSERT INTO AspNetRoles (Id, Name, NormalizedName, ConcurrencyStamp) VALUES
('role-admin-uuid', 'Administrator', 'ADMINISTRATOR', NEWID()),
('role-librarian-uuid', 'Librarian', 'LIBRARIAN', NEWID()),
('role-member-uuid', 'Member', 'MEMBER', NEWID());
```

Use code with caution.

2. Insert Users into AspNetUsers

Next, insert your three default accounts.

Note: The PasswordHash values below are standard valid identity hashes generated for the string Password123.

sql

```
INSERT INTO AspNetUsers (Id, UserName, NormalizedUserName, Email,
NormalizedEmail, EmailConfirmed, PasswordHash, SecurityStamp, ConcurrencyStamp,
PhoneNumberConfirmed, TwoFactorEnabled, LockoutEnabled, AccessFailedCount)
VALUES
-- Administrator
('user-admin-uuid', 'admin@example.com', 'ADMIN@EXAMPLE.COM',
'admin@example.com', 'ADMIN@EXAMPLE.COM', 1,
'AQAAAAIAAYagAAAAEG8Gbi00SYhfjY8...', NEWID(), NEWID(), 0, 0, 1, 0),

-- Librarian
('user-librarian-uuid', 'librarian@example.com', 'LIBRARIAN@EXAMPLE.COM',
'librarian@example.com', 'LIBRARIAN@EXAMPLE.COM', 1,
'AQAAAAIAAYagAAAAEG8Gbi00SYhfjY8...', NEWID(), NEWID(), 0, 0, 1, 0),

-- Member
('user-member-uuid', 'member@example.com', 'MEMBER@EXAMPLE.COM',
'member@example.com', 'MEMBER@EXAMPLE.COM', 1,
'AQAAAAIAAYagAAAAEG8Gbi00SYhfjY8...', NEWID(), NEWID(), 0, 0, 1, 0);
```

Use code with caution.

3. Assign Users to Roles (AspNetUserRoles)

Finally, map each UserId to its corresponding RoleId in the link table:

sql

```
INSERT INTO AspNetUserRoles (UserId, RoleId) VALUES
('user-admin-uuid', 'role-admin-uuid'),      -- Admin account to Admin role
('user-librarian-uuid', 'role-librarian-uuid'), -- Librarian account to
Librarian role
('user-member-uuid', 'role-member-uuid');      -- Member account to Member
role
```

Use code with caution.

Quick Verification Queries

Run these joins to verify everything hooked up perfectly:

sql

```
-- View all system users along with their designated roles
SELECT u.UserName, r.Name AS RoleName
FROM AspNetUsers u
JOIN AspNetUserRoles ur ON u.Id = ur.UserId
JOIN AspNetRoles r ON ur.RoleId = r.Id;
```

```
select * fromAspNetRoles
select * fromAspNetUsers
select * fromAspNetUserRoles
```

Results

Messages

	Id	Name	NormalizedName	ConcurrencyStamp
1	role-admin-uuid	Administrator	ADMINISTRATOR	1637981C-6C1C-4EBA-A164-D8BF61A742E5
2	role-librarian-uuid	Librarian	LIBRARIAN	3971FBB9-2144-426C-A522-CB9DAF34DFE1
3	role-member-uuid	Member	MEMBER	75F9B734-574C-4148-B20B-FE0555740A12

Id	UserName	NormalizedUserName	Email	NormalizedEmail	EmailConfirmed	PasswordHash
1	0b27ce68-a435-498d-a0a9-a253ba806dfe	phalnerupesh89@gmail.com	PHALNERUPESH89@GMAIL.COM	phalnerupesh89@gmail.com	0	AQAAAAIAAYagAAAA
2	user-admin-uuid	admin@example.com	ADMIN@EXAMPLE.COM	admin@example.com	1	AQAAAAIAAYagAAAA
3	user-librarian-uuid	librarian@example.com	LIBRARIAN@EXAMPLE.COM	librarian@example.com	1	AQAAAAIAAYagAAAA
4	user-member-uuid	member@example.com	MEMBER@EXAMPLE.COM	member@example.com	1	AQAAAAIAAYagAAAA

	UserId	RoleId
1	user-admin-uuid	role-admin-uuid
2	user-librarian-uuid	role-librarian-uuid
3	user-member-uuid	role-member-uuid

```
INSERT INTO BOOK (Id, Title, Publisher, PublishYear, Quantity) VALUES
(1, 'bootstrap', 'O Reilly Media', 2022, 5),
(2, 'node js', 'Packt Publishing', 2021, 3),
(3, 'software engineerig', 'McGraw Hill', 2020, 4),
(4, 'sdlc', 'Pearson Education', 2019, 2),
(5, 'stlc', 'Cambridge Press', 2023, 6);
```

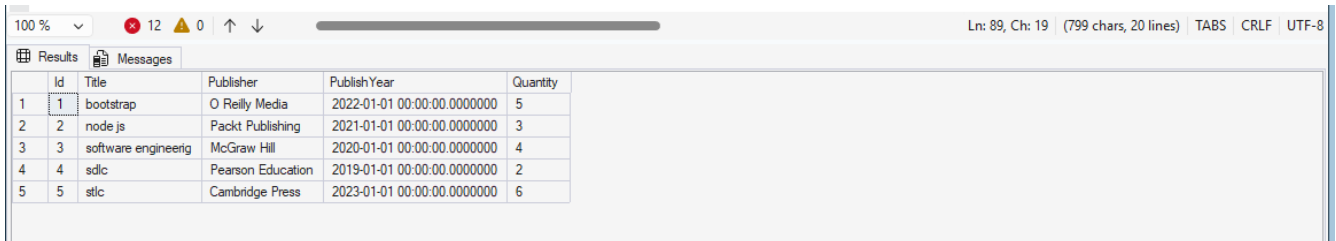
```
SET IDENTITY_INSERT Book ON;
```

```
if data not get insert
solution
```

```
INSERT INTO BOOK (Id, Title, Publisher, PublishYear, Quantity) VALUES
```

```
(1, 'bootstrap', 'O Reilly Media', CAST('2022-01-01' AS datetime2), 5),
(2, 'node js', 'Packt Publishing', CAST('2021-01-01' AS datetime2), 3),
(3, 'software engineerig', 'McGraw Hill', CAST('2020-01-01' AS datetime2), 4),
(4, 'sdlc', 'Pearson Education', CAST('2019-01-01' AS datetime2), 2),
(5, 'stlc', 'Cambridge Press', CAST('2023-01-01' AS datetime2), 6);
```

```
select * from Book
```



100 % 12 0 12 0 12 0 Ln: 89, Ch: 19 | (799 chars, 20 lines) TABS CRLF UTF-8

	Id	Title	Publisher	PublishYear	Quantity
1	1	bootstrap	O Reilly Media	2022-01-01 00:00:00.0000000	5
2	2	node js	Packt Publishing	2021-01-01 00:00:00.0000000	3
3	3	software engineerig	McGraw Hill	2020-01-01 00:00:00.0000000	4
4	4	sdlc	Pearson Education	2019-01-01 00:00:00.0000000	2
5	5	stlc	Cambridge Press	2023-01-01 00:00:00.0000000	6

```
-- 1. Turn OFF the active identity lock on the Book table
SET IDENTITY_INSERT LMSAPP.dbo.Book OFF;

-- 2. Now turn ON the identity insertion permission for the Borrow table
SET IDENTITY_INSERT Borrow ON;

-- 3. Clear out old conflicting rows
DELETE FROM Borrow;

-- 4. Run your exact insert statement safely
INSERT INTO Borrow (Id, BorrowerName, Title, Quantity, BorrowDate, IsApproved, IsReturned)
VALUES
(1, 'member@example.com', 'bootstrap', 1, CAST('2026-07-20' AS datetime2), 1, 0),
(2, 'member@example.com', 'node js', 1, CAST('2026-07-22' AS datetime2), 1, 1),
(3, 'member@example.com', 'sdlc', 1, CAST('2026-07-24' AS datetime2), 0, 0);

-- 5. Turn the lock off for the Borrow table to restore safety defaults
SET IDENTITY_INSERT Borrow OFF;
```

```
SELECT * FROM BOOK
SELECT * FROM Borrow
```


95

8 0

Ln: 93, Ch: 1 (43 chars, 3 lines) TABS CRLF UTF

	id	Title	Publisher	PublishYear	Quantity
1	1	bootstrap	O'Reilly Media	2022-01-01 00:00:00.0000000	5
2	2	node.js	Packt Publishing	2021-01-01 00:00:00.0000000	3
3	3	software engineering	McGraw Hill	2020-01-01 00:00:00.0000000	4
4	4	sdic	Pearson Education	2019-01-01 00:00:00.0000000	2
5	5	stlc	Cambridge Press	2023-01-01 00:00:00.0000000	6

	Id	BorrowerName	Title	Quantity	BorrowDate	IsAproved	IsReturned
1	1	member@example.com	bootstrap	1	2026-07-20 00:00:00.00000000	1	0
2	2	member@example.com	node.js	1	2026-07-22 00:00:00.00000000	1	1
3	3	member@example.com	sdic	1	2026-07-24 00:00:00.00000000	0	0

when we run the application

click sign in

- **Administrator:** admin@example.com / Password123
- **Librarian:** librarian@example.com / Password123
- **Member:** member@example.com / Password123

<https://localhost:7002>
home page

admin controller

<https://localhost:7002/admin/index>

<https://localhost:7002/admin/return>

<https://localhost:7002/admin/BorrowersList>

Books Controllers

<https://localhost:7002/Books/Index>

<https://localhost:7002/Books/Create>

<https://localhost:7002/Books/Borrow>

<https://localhost:7002/Books/details>

<https://localhost:7002/Books/edit>

<https://localhost:7002/Books/delete>

<https://localhost:7002/Books/error>

Area -Identity -Manage

<https://localhost:7002/Identity/Account/Register>

<https://localhost:7002/Identity/Account/Login>

reset pwd

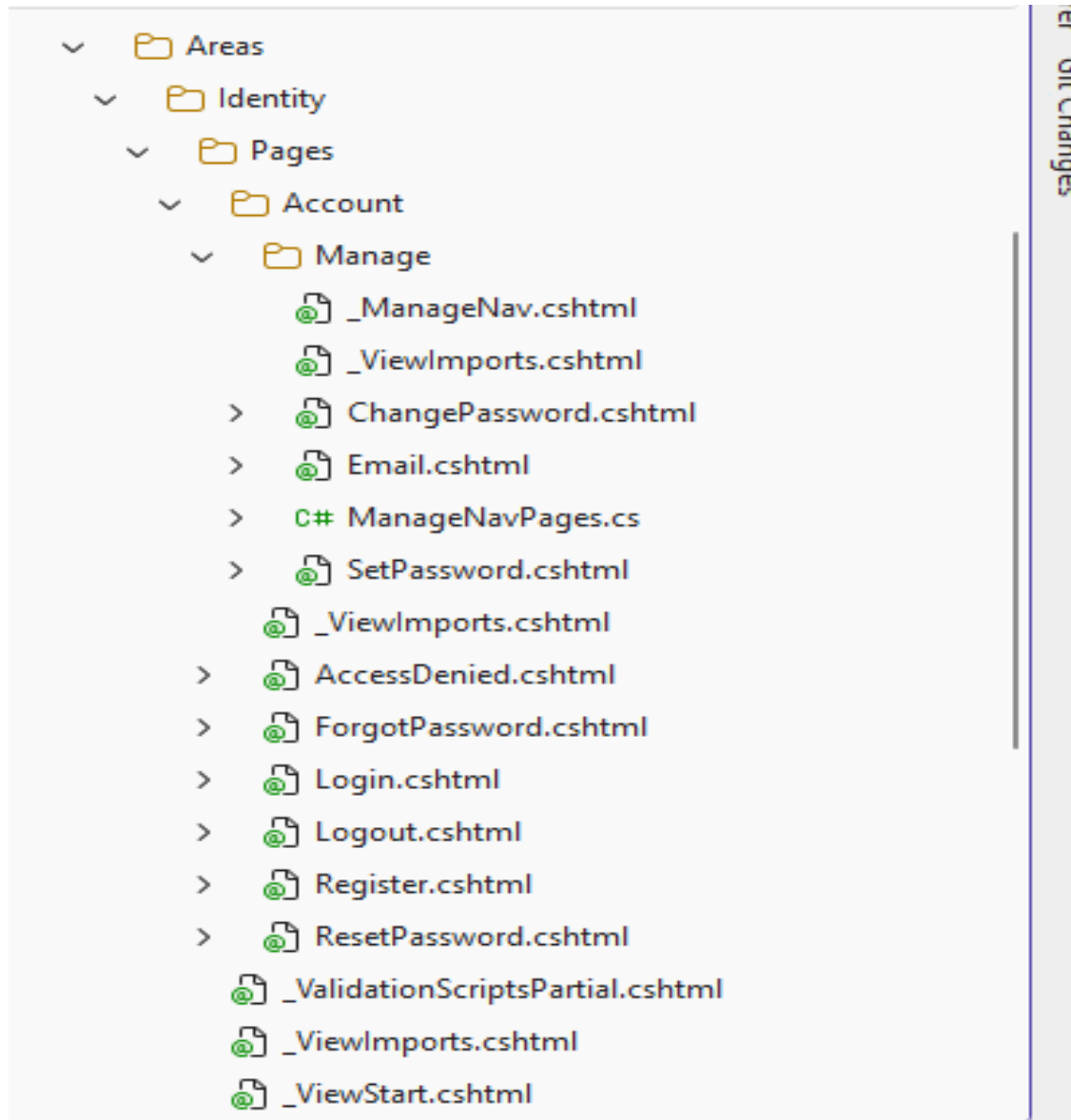
<https://localhost:7002/Identity/Account/ForgotPassword>

register new user

<https://localhost:7002/Identity/Account/Register?returnUrl=%2F>

<https://localhost:7002/Member/Index>

CHECK THIS ONCE



What is ASP.NET Core Identity?

ASP.NET Core Identity is a robust and flexible authentication and authorization system built into ASP.NET Core applications.

It provides everything needed to securely manage user accounts, roles, passwords, and permissions without having to write custom logic from scratch.

Key Features of ASP.NET Core Identity

The following are some of the Key Features of ASP.NET Core Identity:

User Management:

Role Management:

Password Management:

User Authentication and Authorization:

Role-Based Authentication

This means controlling access by assigning users to roles and then allowing or denying access to resources or features based on those roles.

Claims-Based Authentication:

●Beyond roles, claims provide more granular control. For example, a claim might indicate a user's department, age, or subscription level, which can be used to control access to certain features.

Account Security:

Built-in features help protect accounts:

- Account Lockout**
- Two-Factor Authentication (2FA)**
- Email/Phone Verification**

External Login Providers:

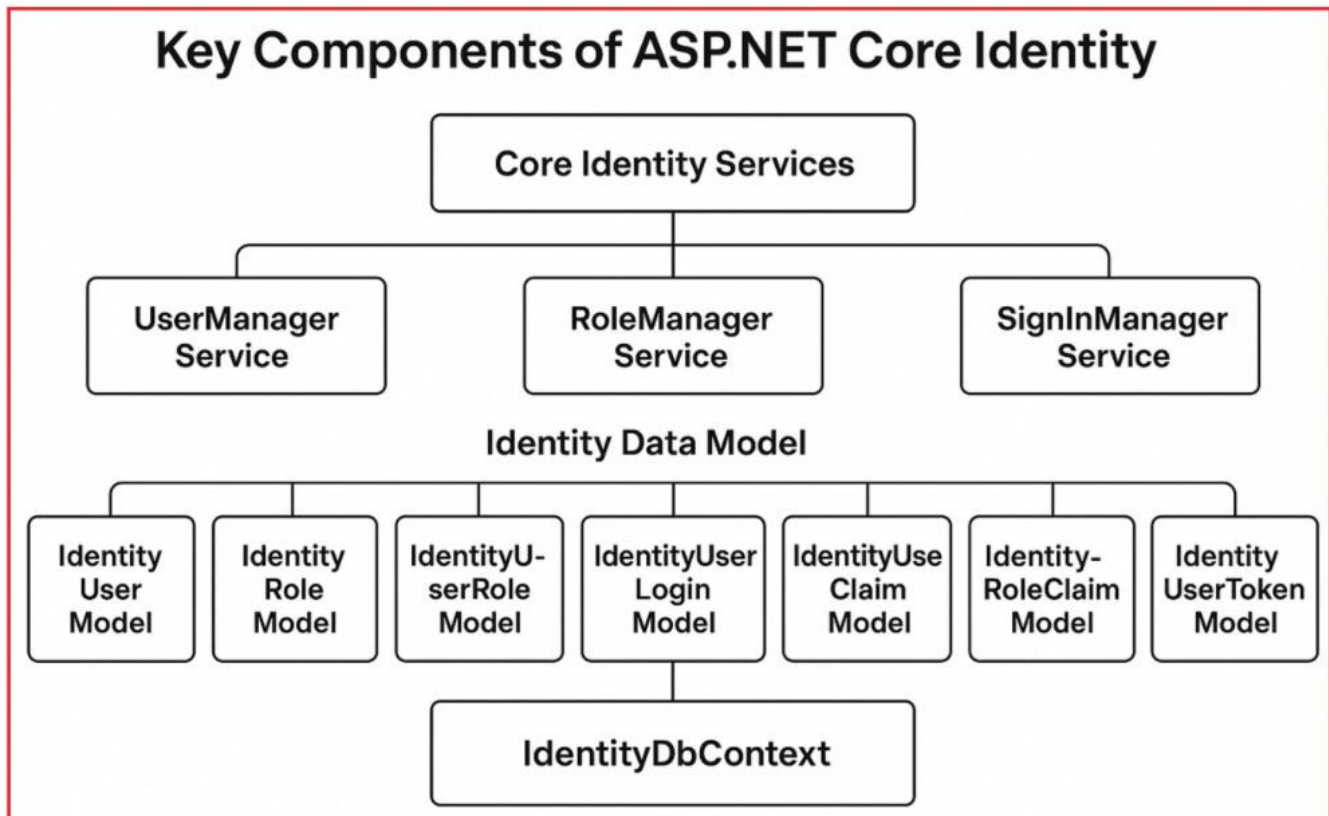
You can allow users to log in using their existing accounts from services like Google or Facebook, making the login process easier and more familiar to them.

Customizability:

● You can extend the built-in user and role classes to add your own fields, such as first name, last name, profile picture URL, or any other information your application needs.

ASP.NET Core Identity Key Components:

The following are the key components of ASP.NET Core Identity:



Core Identity Services:

UserManager Service:

- Ensuring usernames and emails are unique.
 - Enforcing password strength and validation policies.
 - Hashing passwords before storing them ensures that plain-text passwords are never saved.
 - Managing recovery options like password reset tokens and email confirmation tokens.
 - Handling account security features, such as locking accounts after repeated failed login attempts.
- In short, UserManager encapsulates all logic and best practices around user lifecycle management, security, and data integrity.

RoleManager Service

- Updating or deleting roles as needed.
- Retrieving roles for administrative or authorization purposes.

SignInManager Service

- Verifying user credentials (such as username and password) during the sign-in process.
- Managing the authentication session or cookie that keeps users signed in.
- Implementing additional security measures, such as **two-factor authentication (2FA)**, enhances protection.
- Supporting login via external providers like Google, Facebook, or Microsoft accounts.
- Managing the sign-out process to terminate user sessions securely.

Identity Data Model in ASP.NET Core Identity:

Understanding these entities is key to seeing how ASP.NET Core Identity organizes and secures user and role data under the hood. The following are the entity models used in ASP.NET Core Identity. Together, the following models represent the relational structure that supports a flexible, secure, and extensible user authentication and authorization system in ASP.NET Core Identity.

IdentityUser Model

IdentityUser is the fundamental data model that represents a user in the ASP.NET Core Identity system. It stores all the core information about a user, such as:

- Username
- Email address
- Phone number
- Password hash (the encrypted password)
- Account lockout details (to prevent brute-force attacks)
- Other security and profile-related data

This model serves as the base class for users and can be extended with additional properties (like FirstName, LastName, DateOfBirth, LastLogin, etc.) to suit specific application needs. Essentially, every person who registers or is recognized in your application corresponds to one IdentityUser record.

IdentityRole Model

The IdentityRole represents a role within the application. Roles are logical groupings or categories of users, such as “Admin,” “Moderator,” “Customer,” or “Manager”. This model stores basic information about the role, including:

Role name

- Role identifier (ID)
- Optionally, you can extend it to add descriptions or metadata about the role.

Roles help enforce **role-based access control (RBAC)** by grouping permissions and then assigning those roles to users.

IdentityUserRole Model

The IdentityUserRole represents the many-to-many relationship between users and roles. Since a user can have multiple roles and a role can be assigned to multiple users, this model acts as the linking table that associates users with their roles. It stores:

- User ID
- Role ID

This relationship allows ASP.NET Core Identity to determine which roles a particular user belongs to quickly.

IdentityUserLogin Model

The IdentityUserLogin stores information about external login providers linked to a user account. When users log in using third-party services like Google, Facebook, or Microsoft, this model keeps track of:

- The external login provider name (e.g., “Google”, “Facebook”, etc.)
- The unique provider key or user ID from the external provider
- The user in your system that this external login belongs to

This enables **external authentication** and linking multiple external logins to a single local user account.

IdentityUserClaim Model

The IdentityUserClaim represents a claim associated with a specific user. A **claim** is a piece of information about the user, such as their age, email address, membership level, or permission. Claims allow for more fine-grained control than just roles. This model stores:

- The claim type (e.g., “Department”, “Age”, “SubscriptionLevel”)
- The claim value (e.g., “HR”, “30”, “Gold”)
- The user to whom the claim belongs

Claims are useful for **claims-based authorization**, where access decisions depend on these user attributes.

IdentityRoleClaim Model

The IdentityRoleClaim represents claims that are assigned to a role rather than an individual user. Instead of assigning claims to users one by one, you can assign them to roles. Then, all users in that role inherit the claims. This is useful when you want role-based claims, where permissions or attributes are granted to an entire role group. It stores:

- The role ID
- The claim type and claim value associated with that role

IdentityUserToken Model

The IdentityUserToken stores tokens related to a user that are used for security purposes. Tokens can be things like:

- Password reset tokens
- Authentication tokens for two-factor authentication
- Refresh tokens for persistent logins

These tokens are stored securely and linked to the user to help manage authentication flows and security operations that require temporary or revocable tokens.

IdentityDbContext

IdentityDbContext is a special **Entity Framework Core** database context class designed to handle all identity-related data in an ASP.NET Core application. Think of it as the **central manager** that connects your user and role information with your database.

It defines the necessary tables, relationships, and database structure for the Identity system. Using IdentityDbContext, you can easily interact with identity data, like users, roles, claims, and tokens, through Entity Framework Core.

Imagine your application as a hotel, and IdentityDbContext is the front desk manager. Whenever you need to register a guest, assign rooms, handle keys, or check out, the manager knows how to update all the right records (database tables) efficiently.

Why is IdentityDbContext Important?

Predefined Identity Tables: It provides DbSet properties for all the standard identity tables, such as users (IdentityUser), roles (IdentityRole), and their relationships (IdentityUserRole).

●**Database Schema Management:** When you apply Entity Framework migrations, IdentityDbContext creates all the required tables like AspNetUsers, AspNetRoles, and AspNetUserRoles.

●**Customizable:** You can extend IdentityDbContext to add custom fields or relationships to users and roles. You can also add extra tables (DbSets) to support your app's specific needs.

How IdentityDbContext Works in the Application

●When we perform operations like creating a user, assigning roles, or validating credentials, the ASP.NET Core Identity services internally use IdentityDbContext to persist and retrieve data.

●IdentityDbContext abstracts database details and lets you work with identity data using Entity Framework Core.

●You can configure IdentityDbContext with your database connection string, migrations, and other EF Core options.

How Does IdentityDbContext Work in Your Application?

●When your application creates users, assigns roles, or checks credentials, the Identity services use IdentityDbContext behind the scenes to save and retrieve this information from the database.

●It hides the complexity of raw database access, letting you work with identity data easily using Identity service classes.

So, ASP.NET Core Identity streamlines the process of implementing secure user authentication and authorization in modern web applications.

With its customizable models, comprehensive management services, and support for both local and external login providers, it enables developers to build feature-rich, secure, and scalable user management systems efficiently.

ASP.NET Core Identity Setup

Prerequisites to Learn ASP.NET Core Identity

To learn ASP.NET Core Identity effectively, the following are the prerequisites

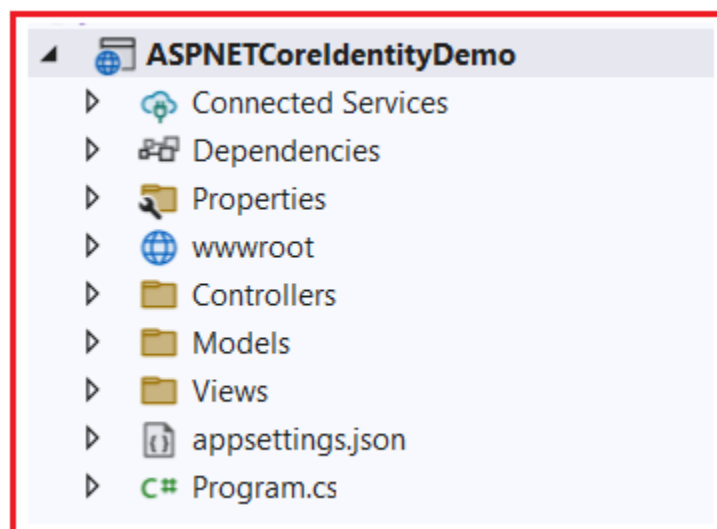
- **Basic Understanding of ASP.NET Core:** Basic understanding of [ASP.NET Core](#), including [MVC \(Model-View-Controller\)](#) architecture. Knowledge of how to create [Web APIs](#) using ASP.NET Core.
- **C# Programming Language:** Good understanding of the [C# Language](#). This is the primary language used for developing ASP.NET Core applications.
- **Entity Framework Core:** Understanding the [Entity Framework Core](#) is Mandatory. EF Core is commonly used for database operations in ASP.NET Core applications. ASP.NET Core Identity uses EF Core to perform database operations.

Step 1: Create a New ASP.NET Core Project

Let us first create a new ASP.NET Core Application using the ASP.NET Core Web App (Model-View-Project) template.

- Use **Visual Studio** or dotnet CLI to create a new **ASP.NET Core Web App (Model-View-Controller)** project.
- Name the project, e.g., **ASPNETCoreIdentityDemo**.
- Select **Authentication Type: None**, as we will manually add Identity ourselves.

This gives you a clean project with the MVC pattern ready to integrate Identity. With this, the project should be created with the following structure.



Step 2: Install Required NuGet Packages

ASP.NET Core Identity needs three packages to be installed in our ASP.NET Core Web application. These packages are as follows:

●**Microsoft.AspNetCore.Identity.EntityFrameworkCore**: This package integrates ASP.NET Core Identity with Entity Framework Core. It provides the necessary implementation to store user, role, and other identity-related data in a database using EF Core.

●**Microsoft.EntityFrameworkCore.SqlServer**: This package is the Entity Framework Core database provider for Microsoft SQL Server. It allows the application to use Entity Framework Core to interact with SQL Server.

●**Microsoft.EntityFrameworkCore.Tools**: This package provides supports for database migration, which help create and update database schemas.

These packages add the core functionality you need to implement Identity backed by SQL Server.

How to install:

Open the Package Manager Console in Visual Studio and then execute the following commands to install the above packages:

●**Install-Package Microsoft.AspNetCore.Identity.EntityFrameworkCore**

●**Install-Package Microsoft.EntityFrameworkCore.SqlServer**

●**Install-Package Microsoft.EntityFrameworkCore.Tools**

Step 3: Define Database Context Class

The **IdentityDbContext** class in ASP.NET Core Identity plays an essential role in integrating the Identity membership management system with Entity Framework Core.

This **IdentityDbContext** class inherits from **DbContext** and is designed to manage the database context for the identity data.

Our application Context class needs to be inherited from the **IdentityDbContext** class instead of the **DbContext** class. This is because **IdentityDbContext** inherits from the Entity Framework Core **DbContext** class and provides all the **DbSet** properties needed to manage the Identity Tables.

First, create a folder named **Data** in the Project root directory. Then, create a class file named **ApplicationDbContext.cs** within the **Data** folder and copy and paste the following code.

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
namespace ASPNETCoreIdentityDemo.Data
{
    public class ApplicationDbContext : IdentityDbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        { }
    }
}
```

} **Note:** This class becomes the EF Core database context responsible for accessing Identity tables. Later, this context will be injected wherever needed.

Step 4: Configuring Connection String in AppSettings.json file:

Next, open the appsettings.json file and copy and paste the following code. Here, we are providing the connection string name **SQLServerIdentityConnection**, which contains the connection string of our SQL server database, where we will create our Identity database.

```
{
  "ConnectionStrings": {
    "SQLServerIdentityConnection": "Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=IdentityCoreDB;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Step 5: Configure ASP.NET Core Identity, Entity Framework Core, and Middleware

To enable authentication and user management in our ASP.NET Core MVC application, we need to configure the Identity services, register the Entity Framework Core context for data access, and add the authentication and authorization middleware into the HTTP request processing pipeline.

Register Entity Framework Core with SQL Server

First, configure EF Core to use SQL Server by registering our custom ApplicationDbContext in the dependency injection container. This context will be responsible for managing our Identity-related database tables.

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("SQLServerIdentityConnection"))
```

Here, the connection string SQLServerIdentityConnection should be defined in the appsettings.json and points to the SQL Server database where Identity data will be stored.

Register ASP.NET Core Identity Services

Next, we need to configure ASP.NET Core Identity services in the DI container. This enables user and role management along with default Identity features such as password hashing and validation.

```
builder.Services.AddIdentity<IdentityUser, IdentityRole>()  
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

IdentityUser and IdentityRole represent the default user and role entities.

AddEntityFrameworkStores<ApplicationDbContext>() instructs Identity to use EF Core context (ApplicationDbContext) for storing user and role data.

Add Authentication and Authorization Middleware

After registering the services, add the middleware components in the HTTP request pipeline to enable authentication and authorization capabilities:

● **app.UseAuthentication();** // Enables the authentication system to validate user credentials

● **app.UseAuthorization();** // Enables authorization checks for access control based on roles or policies

Important Note: Place these middleware calls **after** the routing middleware (app.UseRouting()) but **before** the MVC routing middleware (app.MapControllerRoute(...)) so that the Identity system can authenticate and authorize users **before** requests reach your controllers.

Why this order?

● The app.UseRouting() middleware matches the incoming request to a specific endpoint (e.g., a controller action).

● Authentication (app.UseAuthentication()) needs to happen **after routing** because it uses information about the endpoint and the request to authenticate the user properly.

● Authorization (app.UseAuthorization()) runs **after authentication** and uses the authenticated user information to check whether access to the requested endpoint is allowed based on roles, policies, or claims.

● Finally, app.MapControllerRoute(...) executes the matched endpoint (controller action) only after authentication and authorization are successfully completed.

```
using ASPNETCoreIdentityDemo.Data;  
using Microsoft.AspNetCore.Identity;  
using Microsoft.EntityFrameworkCore;
```

```
namespace ASPNETCoreIdentityDemo  
{  
    public class Program  
    {  
        public static void Main(string[] args)  
        {  
            var builder = WebApplication.CreateBuilder(args);  
  
            // Add services to the container.  
            builder.Services.AddControllersWithViews();
```

```

        // Register Entity Framework Core with SQL Server
        builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("SQLServerIdentityConnection")));

        // Register ASP.NET Core Identity Services
        builder.Services.AddIdentity<IdentityUser, IdentityRole>()
            .AddEntityFrameworkStores<ApplicationDbContext>();

        var app = builder.Build();

        // Configure the HTTP request pipeline.
        if (!app.Environment.IsDevelopment())
        {
            app.UseExceptionHandler("/Home/Error");
            app.UseHsts();
        }

        app.UseHttpsRedirection();
        app.UseStaticFiles();

        app.UseRouting();

        // Add Authentication and Authorization Middleware
        app.UseAuthentication();
        app.UseAuthorization();

        app.MapControllerRoute(
            name: "default",
            pattern: "{controller=Home}/{action=Index}/{id?}");

        app.Run();
    }
}
}

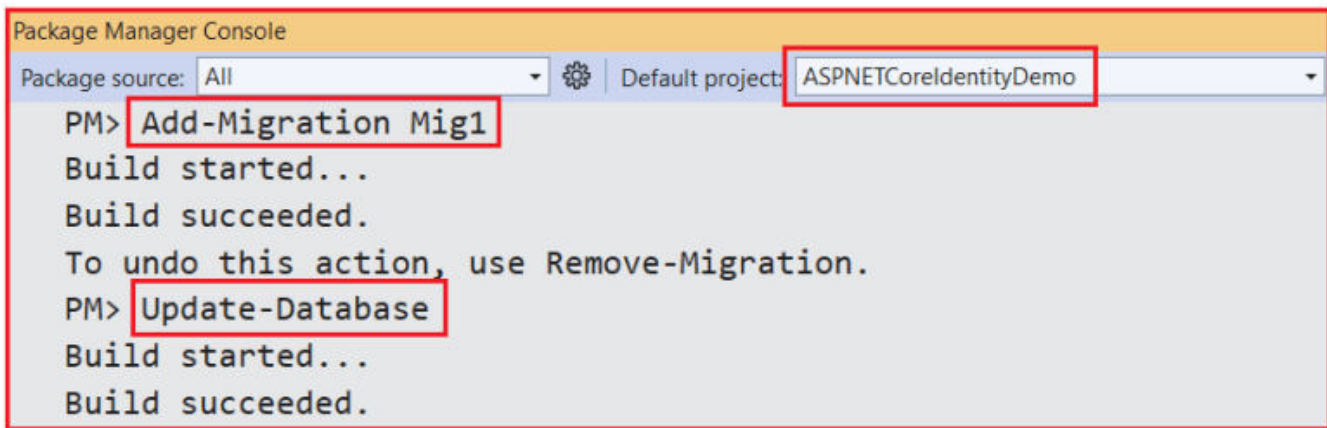
```

This setup ensures:

- EF Core is configured to connect with your database.
- Identity services are wired up to manage users and roles.
- Middleware correctly authenticates and authorizes requests to secure your application.

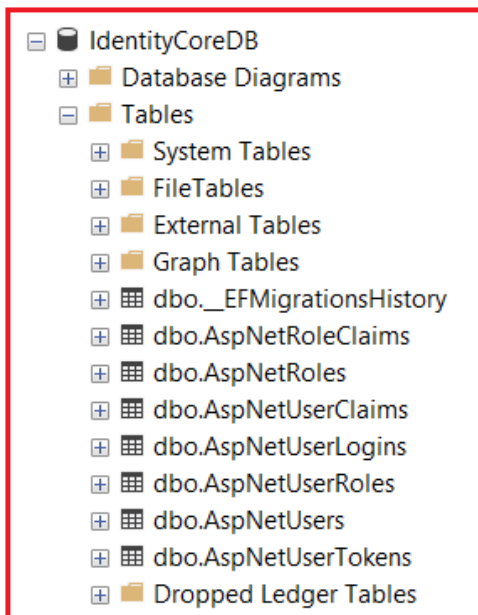
Step 6. Create ASP.NET Core Identity Database Schema

Next, we need to generate the Migration file and update the database. So, open the Visual Studio Package Manager Console and execute the following **Add-Migration** and **Update-Database** commands.



```
Package Manager Console
Package source: All [v] [g] Default project: ASPNETCoreIdentityDemo [v]
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
```

Now, if you verify the database, you should see the following tables in the IdentityCoreDB database.



Differences Between AddIdentity and AddIdentityCore in ASP.NET Core Identity

ASP.NET Core Identity provides two primary methods for registering identity services into the built-in dependency injection (DI) container: **AddIdentity** and **AddIdentityCore**. Although they sound similar, these methods serve different purposes and offer different levels of functionality designed for different application scenarios.

AddIdentity

AddIdentity sets up the **complete and full-featured ASP.NET Core Identity system** in your application.

What it registers:

- User and role management services, including **UserManager**, **RoleManager**, and **SignInManager**.
- Authentication handlers for **cookie-based authentication** are used by default.
- Services for managing security stamps, password hashing, claims, tokens, and lockouts.
- The **default UI supports** common Identity operations such as user registration, login, logout, password recovery, email confirmation, and more (when using Identity UI packages).

Ideal use case:

- Applications that require a **complete user management system with built-in user interface components** for registration, login, and account management. Typically, this applies to **ASP.NET Core MVC web applications** or Razor Pages apps where views and controllers handle the UI.
- Use AddIdentity when you want an **all-in-one, ready-to-use Identity system** with integrated cookie authentication and UI scaffolding.

Syntax to use AddIdentity:

```
// Register ASP.NET Core Identity Services using AddIdentity
builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

AddIdentityCore

AddIdentityCore provides a **minimal and lightweight subset of ASP.NET Core Identity services**, focusing only on the core user and role management features.

What it registers:

- Core services such as **UserManager** and **RoleManager** for managing users and roles.
- Does **NOT** register **SignInManager**, cookie authentication handlers, or UI-related services.
- It provides essential password hashing, user validation, and token providers, but leaves out authentication flow details.

Ideal use case:

- Applications like **API Backends (e.g., ASP.NET Core Web API)**, where authentication is handled differently, commonly via **JWT tokens, OAuth, or other token-based schemes**. In these projects, we often implement our own authentication logic and do not need cookie-based sign-in or built-in UI.
- Use AddIdentityCore when you want **Identity's core user and role features without the extra overhead** of sign-in and UI services. This allows more control over authentication, perfect for **stateless APIs**.

Syntax to use AddIdentityCore:

```
// Register ASP.NET Core Identity Services using AddIdentityCore
builder.Services.AddIdentityCore<IdentityUser>(options =>
{
    // Configure password options, lockout, user settings here if needed
    options.Password.RequireDigit = true;
    options.Password.RequiredLength = 6;
    options.User.RequireUniqueEmail = true;
})
.AddRoles<IdentityRole>() // Optional: add roles support if needed
.AddEntityFrameworkStores<ApplicationDbContext>();
```

Practical Example Scenarios

Scenario 1: I am building a web application in ASP.NET Core MVC and want to quickly enable user registration, login, and password reset with built-in pages and cookies.

●**Use:** AddIdentity

Scenario 2: I am building a REST API that will be consumed by a mobile app and a React frontend. Authentication will use JWT tokens, and the UI is handled entirely by the client.

●**Use:** AddIdentityCore (with JWT authentication)

What is the Default Primary Key Column Type in ASP.NET Core Identity?

●By default, ASP.NET Core Identity uses a **string** type as the primary key for its entities (like users and roles).

●Specifically, the primary key type is string, and the IDs are typically GUIDs (Globally Unique Identifiers) stored as strings.

●For example, the default user ID property in the IdentityUser class is: **public virtual string Id { get; set; }**

●This means that Identity creates tables where the primary key columns (like Id in the AspNetUsers table) are of type **nvarchar(450)** in SQL Server.

Why Use string by Default?

●Using a string-based GUID as the key allows easy generation of unique identifiers without needing a central numeric ID generator.

●It also fits nicely in distributed systems and avoids issues with integer overflow.

●It simplifies integration with external authentication providers (which often use string IDs).

Why Specify IdentityUser and IdentityRole When Registering Identity

Services?

When we register ASP.NET Core Identity in the DI container like this:

```
builder.Services.AddIdentity<IdentityUser, IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>();
```

We are explicitly telling ASP.NET Core **which user and role entity types** our Identity system should use.

Reasons for Specifying These Types

Identity is a Generic Framework. ASP.NET Core Identity is designed to be **extensible and flexible**. It uses generic types to represent the user and role entities:

- **TUser** represents the user entity type.

- **TRole** represents the role entity type.

By specifying `<IdentityUser, IdentityRole>`, we tell the framework:

- Use the built-in `IdentityUser` class for users

- Use the built-in `IdentityRole` class for roles

Allows Customization

If you want to extend the user or role entity with **custom properties** (e.g., adding `FirstName`, `LastName` to the user), you need to create your own classes by inheriting from `IdentityUser` or `IdentityRole`. Then you need to specify your custom classes here, so Identity knows to work with your types. For example:

```
public class ApplicationUser : IdentityUser
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
}

builder.Services.AddIdentity<ApplicationUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

Supports Different Primary Key Types

These classes also define the primary key type for users and roles. For example:

- `IdentityUser` uses a string as the primary key type.

- You might have a custom class like `IdentityUser<int>` if you want an integer key.

Specifying the types makes Identity and EF Core aware of what key type to expect.

ASP.NET Core Identity Tables

AspNetUsers Table:

The **AspNetUsers** table stores all the essential information about the users of your application. It holds their unique identities, login details, security settings, and contact information. This table acts as the primary source for user-related data in the Identity system. The following are the default columns available in this table:

dbo.AspNetUsers
Columns
Id (PK, nvarchar(450), not null)
UserName (nvarchar(256), null)
NormalizedUserName (nvarchar(256), null)
Email (nvarchar(256), null)
NormalizedEmail (nvarchar(256), null)
EmailConfirmed (bit, not null)
PasswordHash (nvarchar(max), null)
SecurityStamp (nvarchar(max), null)
ConcurrencyStamp (nvarchar(max), null)
PhoneNumber (nvarchar(max), null)
PhoneNumberConfirmed (bit, not null)
TwoFactorEnabled (bit, not null)
LockoutEnd (datetimeoffset(7), null)
LockoutEnabled (bit, not null)
AccessFailedCount (int, not null)

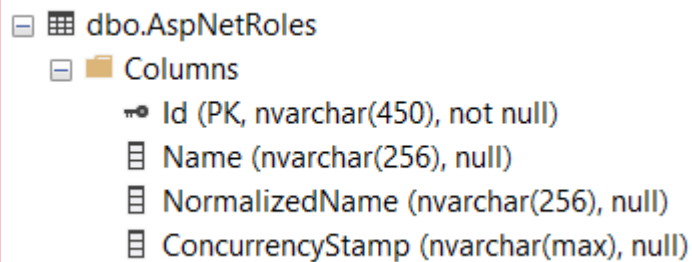
The columns in the AspNetUsers table generally include the following:

- **Id**: Unique identifier for each user (primary key).
- **UserName**: The user's login name.
- **NormalizedUserName**: Uppercase version of UserName for case-insensitive searches.
- **Email**: The user's email address.
- **NormalizedEmail**: Uppercase version of Email for case-insensitive lookups.
- **EmailConfirmed**: Indicates if the user's email is verified.
- **PasswordHash**: Hashed form of the user's password (never stored in plain text).
- **SecurityStamp**: A random value indicates whether any security-related user information has changed. For example, it changes when a user changes their password, resets the password, or adds an external login. Its primary purpose is to invalidate any existing sessions or cookies when security-related information changes. This is a security measure to ensure that the old sessions are no longer valid if credentials are compromised and then changed.
- **ConcurrencyStamp**: A unique value that changes whenever a user profile is updated, used for optimistic concurrency control, ensuring data integrity. In scenarios where multiple attempts to modify the same user record are made simultaneously, ConcurrencyStamp ensures that changes do not conflict.
- **PhoneNumber**: User's phone number.
- **PhoneNumberConfirmed**: Indicates if the phone number is verified.
- **TwoFactorEnabled**: Shows if two-factor authentication is active.
- **LockoutEnd**: Date/time when account lockout ends after failed login attempts.
- **LockoutEnabled**: Indicates if the lockout feature is enabled for the user.
- **AccessFailedCount**: Counts failed login attempts to trigger lockout if necessary.

Relationships: The `AspNetUsers` table is related to most of the other Identity tables. Each user can have multiple roles (via `AspNetUserRoles`), multiple claims (`AspNetUserClaims`), multiple external logins (`AspNetUserLogins`), and multiple tokens (`AspNetUserTokens`). These relationships use the `Id` column as a foreign key in the related tables.

AspNetRoles Table:

The `AspNetRoles` table holds all the defined roles for the application. Roles represent different groups or levels of permissions (e.g., Admin, Moderator, User) that can be assigned to users. The following are the columns available in this table:



dbo.AspNetRoles
Columns
Id (PK, nvarchar(450), not null)
Name (nvarchar(256), null)
NormalizedName (nvarchar(256), null)
ConcurrencyStamp (nvarchar(max), null)

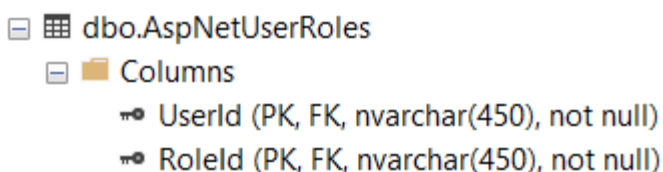
The columns in the `AspNetRoles` table generally include the following:

- **Id:** Unique identifier for each role (usually a GUID).
- **Name:** Human-readable name of the role (e.g., "Admin").
- **NormalizedName:** Uppercase version of the role name for case-insensitive matching.
- **ConcurrencyStamp:** Value that changes on updates, used for concurrency control.

Relationships: The `AspNetRoles` table relates to users through the `AspNetUserRoles` table (assigning roles to users) and to claims through the `AspNetRoleClaims` table (assigning claims/permissions to roles).

AspNetUserRoles Table:

The **AspNetUserRoles** table represents the many-to-many relationship between users and roles. It links users to their respective roles, enabling role-based authorization. The following are the columns available in this table:



dbo.AspNetUserRoles
Columns
UserId (PK, FK, nvarchar(450), not null)
RoleId (PK, FK, nvarchar(450), not null)

The columns in the `AspNetUserRoles` table generally include the following:

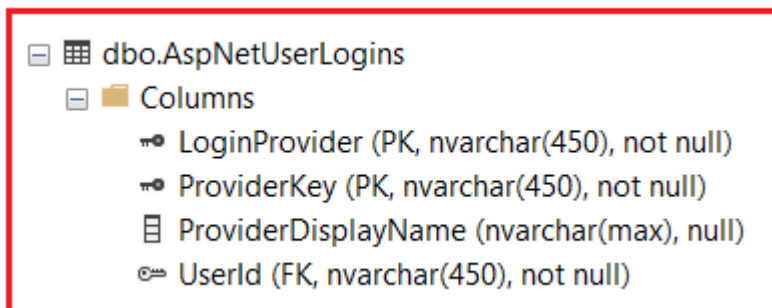
● **UserId:** The user ID from the `AspNetUsers` table. Part of the composite primary key. It corresponds to the `Id` column in the `AspNetUsers` table. It acts as a foreign key linking to the `AspNetUsers` table.

● **RoleId:** The role ID from the `AspNetRoles` table. Part of the composite primary key. It corresponds to the `Id` column in the `AspNetRoles` table. It acts as a foreign key linking to the `AspNetRoles` table.

Relationships: This table connects `AspNetUsers` and `AspNetRoles`, making it possible to look up all roles for a given user or all users for a given role.

AspNetUserLogins Table:

The **AspNetUserLogins** table stores information about external authentication providers that users use to log in, such as Google or Facebook. It links these external logins to local user accounts. The following are the columns available in this table:



dbo.AspNetUserLogins	
Columns	
LoginProvider	(PK, nvarchar(450), not null)
ProviderKey	(PK, nvarchar(450), not null)
ProviderDisplayName	(nvarchar(max), null)
UserId	(FK, nvarchar(450), not null)

The columns in the `AspNetUserLogins` table generally include the following:

● **LoginProvider:** This column stores the name of the external authentication provider (e.g., Google, Facebook, Microsoft, etc.). It is part of the composite primary key for the table.

● **ProviderKey:** This column stores the unique identifier provided by the external login provider for the user. For example, when a user logs in using Google, this field will store the unique ID assigned to the user by Google. It is also part of the composite primary key.

● **ProviderDisplayName:** This optional column can store the display name of the login provider (e.g., “Google” instead of “google.com”). It is mainly used for display purposes in the UI.

● **UserId:** The local user ID that is linked to this login. It acts as a foreign key linking to the `Id` column in the `AspNetUsers` table.

Relationships: Each entry in this table connects a user (`UserId`) to an external authentication provider, allowing users to log in using social logins.

AspNetUserTokens Table:

The `AspNetUserTokens` table stores tokens for users that can be used for various purposes, like password reset, email confirmation, or two-factor authentication. The following are the columns available in this table:

dbo.AspNetUserTokens
Columns
- UserId (PK, FK, nvarchar(450), not null) - LoginProvider (PK, nvarchar(450), not null) - Name (PK, nvarchar(450), not null) - Value (nvarchar(max), null)

The columns in the AspNetUserTokens table generally include the following:

●**UserId:** The User ID from the AspNetUsers table.

●**LoginProvider:** Name of the provider that generated the token.

●**Name:** This column stores the name of the token, such as an email confirmation token, password reset token, or two-factor authentication token. It could be something like PasswordReset, EmailConfirmation, AccessToken, etc.

●**Value:** The value of the token.

Relationships: Each token entry is tied to a user and is typically used for security operations or workflow steps that require a one-time token.

AspNetUserClaims Table:

The **AspNetUserClaims** table stores additional claims tied to individual users. Claims are key-value pairs that hold extra information about the user, often used in authorization. Each claim is a key-value pair associated with a user and can represent various attributes such as email address, full name, role, or any other custom data relevant to the application. The following are the columns available in this table:

dbo.AspNetUserClaims
Columns
- Id (PK, int, not null) - UserId (FK, nvarchar(450), not null) - ClaimType (nvarchar(max), null) - ClaimValue (nvarchar(max), null)

The columns in the AspNetUserClaims table generally include the following:

●**Id:** The primary key for the user claim. It is an integer.

●**UserId:** The ID of the user associated with this claim. It acts as a foreign key linking to the Id column in the AspNetUsers table.

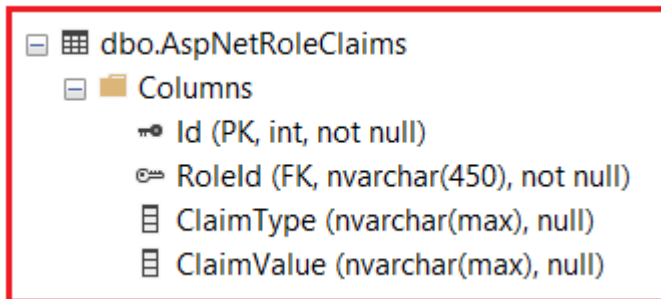
●**ClaimType:** The type of the claim (e.g., "birthdate").

●**ClaimValue:** The value of the claim (e.g., "1980-01-01").

Relationships: Each claim entry links to a user and defines additional properties or access rights unique to that user.

AspNetRoleClaims Table:

The **AspNetRoleClaims** table stores claims associated with roles. These claims apply to all users who are assigned that role, enabling permission-based claims at the role level. The following are the columns available in this table:



dbo.AspNetRoleClaims
Columns
Id (PK, int, not null)
RoleId (FK, nvarchar(450), not null)
ClaimType (nvarchar(max), null)
ClaimValue (nvarchar(max), null)

The columns in the AspNetRoleClaims table generally include the following:

●**Id:** Unique integer identifier for the role claim.

●**RoleId:** The role ID associated with this claim. A foreign key that links to the Id column in the AspNetRoles table.

●**ClaimType:** This column stores the type of the claim, such as Permission, AccessLevel, or any other type that makes sense in the context of your application.

●**ClaimValue:** The actual value of the claim. For example, if the claim type is **Permission**, the claim value might be **Edit_User** or **View_Reports**, etc.

Relationships: Each claim entry is associated with a specific role. All users in that role will inherit the claims defined here.

Note: The structure of the tables is consistent across different database providers (e.g., SQL Server, PostgreSQL, MySQL), but there might be minor differences in types or constraints based on the database system used.

UserManager SignInManager RoleManager in ASP.NET Core Identity

UserManager Class in ASP.NET Core Identity

The **userManager<TUser>** class is used to manage user information. The TUser parameter is typically an instance of a class that extends **IdentityUser**. This class provides methods for creating, deleting, updating, and retrieving user information and managing user-related features like changing passwords, assigning roles, and claims. The following are some of the key functionalities provided by UserManager:

●**Creating and Deleting Users:** You can create new user accounts using CreateAsync and delete them with DeleteAsync.

●**Managing Passwords:** This includes changing passwords (ChangePasswordAsync), resetting passwords (ResetPasswordAsync), and checking if a password meets the specified criteria (CheckPasswordAsync).

●**Managing User Roles:** Functions like `AddToRoleAsync` and `RemoveFromRoleAsync` are used to assign and remove roles for users.

●**Lockout functionality:** This includes enabling or disabling account lockout and setting lockout durations.

●**Two-factor authentication:** Managing two-factor authentication settings for users.

How to Use UserManager Class in ASP.NET Core Identity?

The `UserManager` in ASP.NET Core Identity is a class that provides methods for managing users in your application. The `UserManager` is a generic class and works with an entity (`TUser`) that must extend `IdentityUser`. Typically, it is injected into your controllers or other services via dependency injection. For a better understanding, please observe the following code where we have injected the `UserManager<ApplicationUser>` instance through the constructor of a controller. Here, `ApplicationUser` is a class inherited from the `IdentityUser` class.

```
public class MyController : Controller
{
    private readonly UserManager<IdentityUser> _userManager;
    public MyController(UserManager<ApplicationUser> userManager)
    {
        _userManager = userManager;
    }
}
```

Methods Provided by UserManager Class in ASP.NET Core Identity:

●**CreateAsync(TUser user, string password):** Creates a new user with the specified password.

●**DeleteAsync(TUser user):** Deletes the specified user.

●**FindByIdAsync(string userId):** Finds and returns a user, if any, who has the specified user ID.

●**FindByEmailAsync(string email):** Finds and returns a user, if any, who has the specified email address.

●**FindByNameAsync(string userName):** Finds and returns a user, if any, who has the specified username.

●**UpdateAsync(TUser user):** Updates the specified user's information.

●**AddToRoleAsync(TUser user, string role):** Adds the specified user to the named role.

●**RemoveFromRoleAsync(TUser user, string role):** Remove the specified user from the named role.

●**IsInRoleAsync(TUser user, string role):** Returns true if the specified user is in the named role.

●**ChangePasswordAsync(TUser user, string currentPassword, string newPassword):** Changes a user's password after confirming the current password.

●**ResetPasswordAsync(TUser user, string token, string newPassword):** After confirming the password reset token, it resets the user's password to the specified new password.

●**GeneratePasswordResetTokenAsync(TUser user):** Generates a password reset token for the user.

●**ConfirmEmailAsync(TUser user, string token):** Confirms the user's email address with the specified token.

- GenerateEmailConfirmationTokenAsync(TUser user)**: Generates an email confirmation token for the user.
- CheckPasswordAsync(TUser user, string password)**: This function checks if the given password is valid for the specified user.
- GetRolesAsync(TUser user)**: Returns a list of role names the specified user belongs to.
- GetUserIdAsync(TUser user)**: Returns the user ID for the specified user.
- GetUserNameAsync(TUser user)**: Returns the username for the specified user.
- GetUsersInRoleAsync(string roleName)**: Returns a list of users who belong to the specified role.
- AccessFailedAsync(TUser user)**: This function increments the user's access failed count, and if it reaches the maximum number of attempts, it locks out the user.

SignInManager Class in ASP.NET Core Identity

The `SignInManager<TUser>` in ASP.NET Core Identity is a generic class that handles authentication and sign-in operations for users. It depends on the `userManager` class to manage user authentication and perform sign-in and sign-out operations. The following are some of its primary responsibilities:

- Password Sign-in**: The `PasswordSignInAsync` method of the `SignInManager<TUser>` class is used to sign in a user with their username and password.
- External Authentication**: The `ExternalLoginSignInAsync` method of the `SignInManager<TUser>` class handles sign-ins with external login providers like Google, Facebook, etc.
- Sign-out**: `SignOutAsync` method is used to sign a user out of the application.
- Two-Factor Authentication Sign-in**: Methods like `TwoFactorSignInAsync` manages the process of signing in with two-factor authentication.
- Remember Me Functionality**: This is handled during the sign-in process, where a persistent cookie will be created if the user opts for the "remember me" option.

How to Use ASP.NET Core Identity SignInManager?

`SignInManager` instance is typically injected into your controllers or services using dependency injection. For example, in the controller, add a constructor that accepts `SignInManager<IdentityUser>` as a parameter as follows:

```
private readonly SignInManager<IdentityUser> _signInManager;
public AccountController(SignInManager<IdentityUser> signInManager)
{
    _signInManager = signInManager;
}
```

Methods Provided by SignInManager Class in ASP.NET Core Identity:

- PasswordSignInAsync(TUser user, string password, bool isPersistent, bool lockoutOnFailure)**: Attempts to sign in a user with a username and password. The `isPersistent` parameter indicates whether the sign-in cookie should persist across browser sessions, and `lockoutOnFailure` indicates if the user account should be locked on a failed sign-in attempt.

- **SignInAsync():** Signs the current user out.
- **RefreshSignInAsync(TUser user):** This function refreshes the current user's sign-in information, which is typically used when updating the security stamp.
- **SignInAsync(TUser user, bool isPersistent, string authenticationMethod = null):** Signs in the user without password verification.
- **ExternalLoginSignInAsync(string loginProvider, string providerKey, bool isPersistent):** Attempts to sign in a user with an external login (such as Facebook or Google).
- **GetExternalLoginInfoAsync(string expectedXsrf = null):** Gets the information about the user from an external login provider.
- **TwoFactorSignInAsync(string provider, string code, bool isPersistent, bool rememberClient):** Attempts to sign in a user with a two-factor authentication code.
- **IsSignedIn(ClaimsPrincipal principal):** Checks if the specified ClaimsPrincipal is signed in.
- **ValidateSecurityStampAsync(TUser user):** Validate the security stamp for the specified user.
- **CanSignInAsync(TUser user):** Checks if the specified user is allowed to sign in.
- **CheckPasswordSignInAsync(TUser user, string password, bool lockoutOnFailure):** Attempts to sign in a user with a password without generating a security stamp.
- **ConfirmEmailSignInAsync(TUser user, string token, bool isPersistent):** Signs in a user after confirming their email.
- **GetTwoFactorAuthenticationUserAsync():** Retrieves the user for whom two-factor authentication is currently active.
- **RememberTwoFactorClientAsync(TUser user):** This function remembers the browser/device for two-factor authentication to avoid requesting a second factor on subsequent logins.
- **GetRememberedTwoFactorClientAsync():** Retrieves the information about a remembered browser/device for two-factor authentication.

RoleManager Class in ASP.NET Core Identity

The `RoleManager<TRole>` class manages roles in the application. The `TRole` parameter is typically an instance of a class that extends `IdentityRole`. This class provides methods for the creation, deletion, and updating of roles, assigning roles to users, as well as managing claims associated with roles. This is used to implement role-based access control (RBAC) in your applications to manage user permissions effectively. The following are some of the key functionalities of the `RoleManager` class in ASP.NET Core Identity:

- **Creating and Deleting Roles:** Methods like `CreateAsync` and `DeleteAsync` methods allow for managing roles within the application.
- **Role Existence Checks:** `RoleExistsAsync` checks whether a specified role exists in the system.
- **Managing Role Claims:** The `AddClaimAsync` and `RemoveClaimAsync` methods add or remove claims to roles, facilitating fine-grained access control based on role capabilities.
- **Adding and Removing Users from Roles:** `AddToRoleAsync` and `RemoveRoleAsync` methods allow for Adding and removing users from roles.

How to use ASP.NET Core Identity RoleManager Class?

Inject the RoleManager instance into your classes (like Controllers or service classes) where you need to manage roles. This is done through dependency injection. For example, in the controller, add a constructor that accepts RoleManager<IdentityRole> as a parameter as follows:

```
private readonly RoleManager<IdentityRole> _roleManager;
public MyController(RoleManager<IdentityRole> roleManager)
{
    _roleManager = roleManager;
}
```

Methods Provided by RoleManager Class in ASP.NET Core Identity:

- **CreateAsync(T role)**: Creates a new role asynchronously.
- **UpdateAsync(T role)**: Updates a role asynchronously.
- **DeleteAsync(T role)**: Deletes a role asynchronously.
- **FindByIdAsync(string roleId)**: Finds a role by its ID asynchronously.
- **FindByNameAsync(string roleName)**: Finds a role by its name asynchronously.
- **RoleExistsAsync(string roleName)**: Checks if a role exists asynchronously.
- **AddClaimAsync(T role, Claim claim)**: Adds a claim to a role asynchronously.
- **RemoveClaimAsync(T role, Claim claim)**: Removes a claim from a role asynchronously.
- **GetClaimsAsync(T role)**: Gets claims for a role asynchronously.
- **GetRolesAsync(TUser user)**: Gets the roles for the specified user asynchronously.

Differences Between UserManager, SignInManager, and RoleManager in

ASP.NET Core Identity

- **UserManager** deals with users, **SignInManager** handles the authentication process, and **RoleManager** focuses on roles.
- **SignInManager** often depends on **UserManager** to perform user-related checks during the sign-in process, but **UserManager** can function independently for user management tasks.
- **SignInManager** is more about authentication (verifying who the user is), while **UserManager** and **RoleManager** can be involved in authorization (determining what the user is allowed to do). **RoleManager** specifically deals with role-based access control.
- **SignInManager** contains specific security features, such as external login and two-factor authentication (2FA), that are not part of **UserManager** or **RoleManager**.

In a controller, you might inject these managers as follows:

```
public class AccountController : Controller
{
```

```

private readonly UserManager<IdentityUser> _userManager;
private readonly SignInManager<IdentityUser> _signInManager;
private readonly RoleManager<IdentityRole> _roleManager;
public AccountController(
    UserManager<IdentityUser> userManager,
    SignInManager<IdentityUser> signInManager,
    RoleManager<IdentityRole> roleManager)
{
    _userManager = userManager;
    _signInManager = signInManager;
    _roleManager = roleManager;
}
// Controller actions...
}

```

These managers simplify the process of implementing common identity and access management tasks in ASP.NET Core applications. From our next article onwards, we will use these Manager classes to perform different kinds of operations.